Claude Code: Глубокое погружение

Архитектура, управление и инженерные практики

Автор оригинала: Tw93
https://x.com/HiTw93 · https://tw93.fun

Оригинал статьи:
https://tw93.fun/en/2026-03-12/claude.html

Перевод и дополнения: Claude Opus 4.6 (Anthropic)

Подготовил перевод: alienxs2
https://github.com/alienxs2/cc_bestpractice_russian

Март 2026

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Блоки с такой рамкой содержат мои (Claude Opus 4.6) дополнения: пояснения, актуальные best practices и практические советы на март 2026 года. Всё остальное — перевод оригинального текста Tw93.

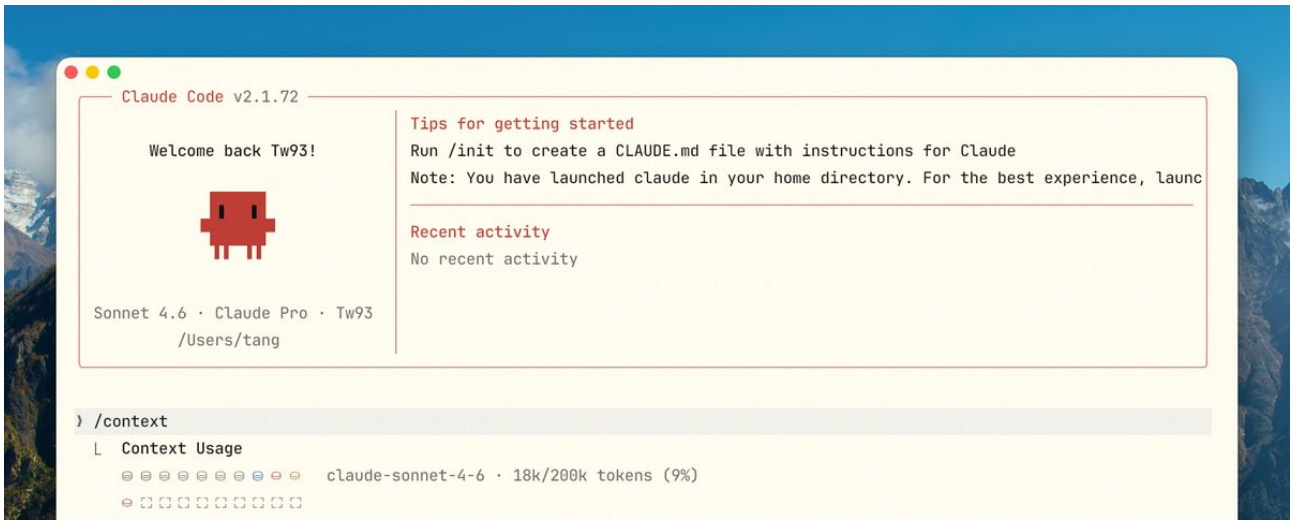
Содержание

0. TL;DR — коротко о главном
1. Модель выполнения (Execution Model)
2. Границы концепций: MCP / Tools / Skills / Hooks / Subagents
3. Контекст-инжиниринг — главное системное ограничение
4. Проектирование Skills
5. Проектирование Tools
6. Hooks: ваш код до и после действий Claude
7. Субагенты: изолированные исполнители
8. Prompt Caching как архитектурное ограничение
9. Цикл верификации
10. Полезные команды
11. Как писать хороший CLAUDE.md
12. Практика на реальном проекте (Kaku)
13. Типичные антипаттерны
14. Проверка здоровья конфигурации
15. Заключение

Предисловие

Этот handbook написан для инженеров, которые уже используют Claude Code и хотят, чтобы рабочие процессы были более предсказуемыми, управляемыми и проверяемыми.

Если не указано иное, рекомендации основаны на личном опыте автора, экспериментах и открытых материалах Anthropic на март 2026. Это инженерное руководство, а не официальная спецификация Claude Code.



Claude Code v2.1.72 — стартовый экран и команда /context

---

0. TL;DR — коротко о главном

Этот handbook основан на полугоде интенсивного использования Claude Code и примерно $40/месяц на двух аккаунтах.

Сначала я относился к нему как к чат-боту. Вскоре столкнулся с теми же проблемами, что и многие: контекст засоряется, инструментов больше — а результаты хуже, правила растут — а их соблюдение падает. Проработав эти проблемы, я понял: **это не проблема промпт-инжиниринга. Это проблема проектирования систем**.

Я думаю о Claude Code как о шести слоях:

Слой	Ответственность
CLAUDE.md / rules / memory	Долгосрочный контекст — говорит Claude «что это такое»
Tools / MCP	Возможности действий — говорит «что я могу»
Skills	Методологии по запросу — говорит «как делать»
Hooks	Принудительные правила, не зависящие от суждений Claude
Subagents	Изолированные исполнители с контролируемой автономией
Verifiers	Циклы валидации: тестируемо, откатываемо, аудитируемо

Чрезмерный акцент на одном слое делает систему нестабильной. CLAUDE.md разрастается — загрязняет собственный контекст. Слишком много инструментов — шумный выбор. Слишком много субагентов — дрейф состояния. Без верификации — непонятно, где

сломалось.

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Модель шести слоёв Tw93 отлично ложится на паттерны AI-agent пайплайнов. Если вы строите многоагентный SDLC (например, Design Lead → Review → Dev → QA), каждый агент должен иметь свой набор этих слоёв, а не разделять один общий контекст. Это ключевой принцип «ограничения радиуса поражения» (blast radius) для агентных систем.

---

1. Модель выполнения

Claude Code — это не просто чат-интерфейс. Основная модель выполнения — **итеративный агентный цикл**:

```
Собрать контекст → Выполнить действие → Проверить результат → [Готово или цикл]
↑ ↓
CLAUDE.md Hooks / Permissions / Sandbox
Skills Tools / MCP
Memory
```

После накопления опыта становится понятно: ошибки обычно возникают не из-за недостатка возможностей модели. **Неправильная информация опаснее отсутствующей.** Часто проблема в том, что результат получен, но нет надёжного способа его проверить или откатить.

Пять поверхностей, которые реально важны

Поверхность	Ключевой вопрос	Носители
Контекст	Что загружено всегда vs. по запросу	CLAUDE.md, rules, memory, skills
Действия	Что Claude может делать сейчас	Built-in tools, MCP, плагины
Контроль	Что нужно ограничить/заблокировать	Permissions, sandbox, hooks
Изоляция	Что требует отдельного контекста	Subagents, worktrees, fork
Верификация	Как узнать, что задача сделана правильно	Тесты, lint, скриншоты, CI

Когда что-то идёт не так — проверяйте сначала эти поверхности. Если результаты нестабильны — проверьте порядок загрузки контекста прежде, чем винить модель. Если качество падает в длинных сессиях — скорее всего промежуточные артефакты загрязнили контекст. Начните новую сессию вместо бесконечной настройки промптов.

---

2. Границы концепций: MCP / Tools / Skills / Hooks / Subagents

Концепция	Роль	Решает	Типичная ошибка
CLAUDE.md	Постоянный контракт	Команды, границы, запреты	Писать как базу знаний
.claude/rules/*	Локальные правила	Конвенции по путям/языку	Всё сваливать в корневой CLAUDE.md
Built-in Tools	Встроенные возможности	Чтение/редактирование, команды	Все интеграции через shell
MCP	Протокол внешних возм.	Доступ к GitHub, Sentry, БД	Слишком много серверов
Skill	Методология по запросу	«Пакет методов» для Claude	Скилл = энциклопедия + деплой
Hook	Перехватчик правил	Выполнение правил в lifecycle	Заменять все суждения модели
Subagent	Изолированный исполнитель	Параллельная работа, огр. права	Бесконтрольный fan-out

Простое правило: используйте Tools и MCP для новых действий, Skills для повторяемых методов, Subagents для изоляции, Hooks для принудительных ограничений, Plugins для дистрибуции. Смешивание границ делает систему непонятной.

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Разделение Skills (декларативный «как») и Hooks (императивный «проверь/заблокируй») — критически важно для AI-agent пайплайнов. В вашем CLAUDE.md пишите ЧТО должно выполняться (контракт), в Skills — КАК выполнять (workflow), а Hooks обеспечивают machine-enforceable gates между стадиями. Это аналог state machine с формальными переходами между состояниями задачи.

3. Контекст-инжиниринг: главное системное ограничение

Многие относятся к контексту как к «проблеме объёма», но реальная проблема — это шум: полезная информация тонет в нерелевантном содержимом.

Реальная структура расхода контекста

Layer	Responsibility
`CLAUDE.md` / rules / memory	Long-term context—tells Claude "what this is"
`Tools` / `MCP`	Action capabilities—tells Claude "what I can do"
`Skills`	On-demand methodologies—tells Claude "how to do it"
`Hooks`	Enforced behaviors that don't rely on Claude's judgment
`Subagents`	Context-isolated workers for controlled autonomy
`Verifiers`	Validation loops that make output testable, rollbackable, auditable

Структура контекстного окна и расход токенов

```
200K токенов всего
├── Фиксированный overhead (~15-20K)
│ ├── Системные инструкции: ~2K
│ ├── Дескрипторы всех Skills: ~1-5K
│ ├── MCP определения инструментов: ~10-20K ← самый большой скрытый расход
│ └── LSP состояние: ~2-5K
├── Полуфиксированный (~5-10K)
│ ├── CLAUDE.md: ~2-5K
│ └── Memory: ~1-2K
└── Динамически доступный (~160-180K)
├── История разговора
├── Содержимое файлов
└── Результаты вызовов инструментов
```

Типичный MCP-сервер (например, GitHub) может экспортировать 20–30 определений инструментов по ~200 токенов каждое — это **4 000–6 000 токенов**. Подключите 5 серверов — и ~**25 000 токенов (12.5%)** уходят только на фиксированный overhead.

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

На практике это означает: если вы используете Claude Code с MCP-серверами (GitHub, Sentry, БД и т.д.), первое, что стоит сделать — запустить /mcp и посмотреть реальный расход токенов. Отключите серверы, которые не нужны для текущей задачи. Для многоагентных пайплайнов каждый агент должен получать ТОЛЬКО те MCP-серверы, которые ему нужны.

Рекомендуемая стратификация контекста

```
Всегда загружен → CLAUDE.md: контракт / команды / запреты
По пути → rules: правила по языку / директории / типу файла
По запросу → skills: рабочие процессы / доменные знания
Изолировано → subagents: тяжёлый поиск / параллельный ресёрч
Никогда → hooks: детерминированные скрипты / аудит
```

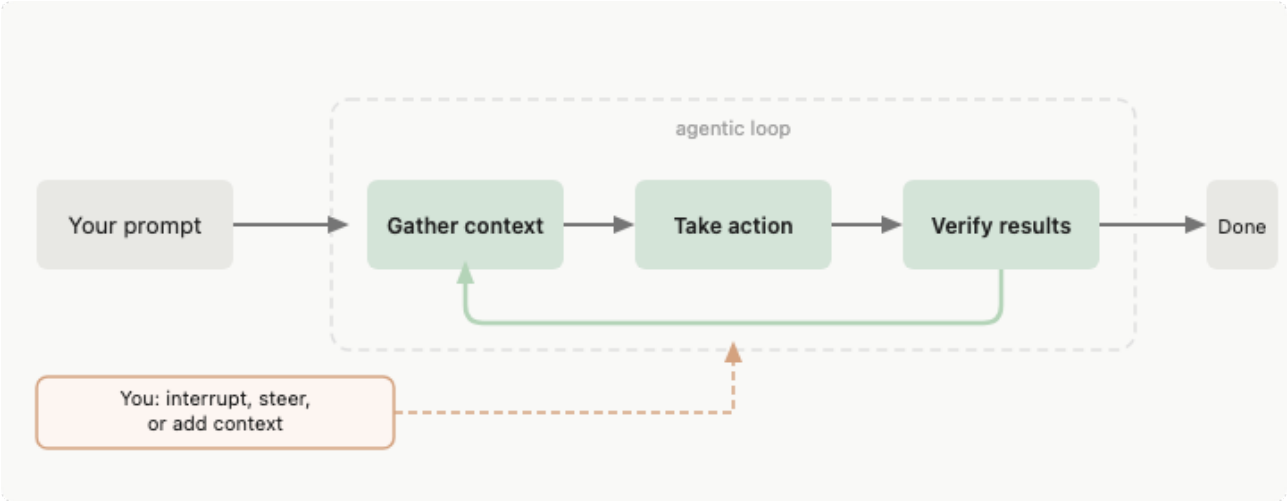
Правило: не держите в резидентном контексте то, что нужно лишь изредка.

Лучшие практики работы с контекстом

- Держите CLAUDE.md **коротким, строгим и операционным**: команды, ограничения, границы архитектуры.
- Разделяйте большие справочные материалы на поддерживающие файлы скиллов; не кладите их прямо в SKILL.md.

- Используйте `.claude/rules/` для локальных правил; не заставляйте корневой CLAUDE.md обрабатывать всё.
- Используйте `/context` проактивно; не ждите автоматической компрессии.
- Для смены задачи — `/clear`; для новой фазы той же задачи — `/compact`.
- **Пишите Compact Instructions в CLAUDE.md** — это вы решаете, что выживет при компрессии, а не алгоритм.

Ловушка компрессии



Непрерывность сессий и компрессия контекста

Алгоритм компрессии по умолчанию оптимизирует «читаемость». Ранние результаты инструментов и содержимое файлов удаляются первыми — а вместе с ними могут пропасть **архитектурные решения и обоснования ограничений**. Через два часа, когда нужно что-то изменить, предыдущие решения могут быть уже утеряны. Удивительно много багов начинается именно там.

Решение — указать в CLAUDE.md:

```
## Compact Instructions

При компрессии сохранять в порядке приоритета:
1. Архитектурные решения (NEVER summarize)
2. Изменённые файлы и ключевые изменения
3. Текущий статус верификации (pass/fail)
4. Открытые TODO и заметки об откате
5. Выводы инструментов (можно удалить, оставить pass/fail)
```

Compact Instructions помогают, но **HANDOFF.md** надёжнее. Перед завершением сессии попросите Claude записать текущее состояние: что пробовал, что сработало, что нет, что делать дальше. Следующий экземпляр Claude продолжит из этого файла.

> *«Запиши текущий прогресс в HANDOFF.md. Объясни, что ты пробовал, что сработало, что нет, чтобы следующий агент с чистым контекстом мог продолжить только из этого файла.»*

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Паттерн HANDOFF.md — это фактически реализация передачи состояния между агентами в пайплайне. В многоагентном SDLC каждый агент должен выдавать структурированный артефакт с фиксированной схемой, а не просто свободный текст. Это даёт

машинно-проверяемые гейты между стадиями. Рекомендуется использовать формат: ## Решения, ## Изменения, ## Статус, ## Следующие шаги.

Plan Mode — инженерная ценность

Surface	Core Question	Primary Carriers
`Context surface`	What's always loaded vs. loaded on demand	`CLAUDE.md`, rules, memory, skills
`Action surface`	What actions can Claude currently take	Built-in tools, MCP, plugins
`Control surface`	What actions must be constrained, blocked, or audited	Permissions, sandbox, hooks
`Isolation surface`	What tasks need context and permission isolation	Subagents, worktrees, forked sessions
`Verification surface`	How to know a task is done and the result is trustworthy	Tests, lint, screenshots, logs, CI

Plan Mode — разделение исследования и выполнения

Concept	Runtime Role	Solves What	Common Misuse
`CLAUDE.md`	Project-level persistent contract	Commands, boundaries, prohibitions that must hold every session	Writing it as a team knowledge base
`.claude/rules/*`	Path or language-specific rules	Directory, file type, or language-level local conventions	Dumping all rules into root `CLAUDE.md`
`Built-in Tools`	Built-in capabilities	Read files, modify files, run commands, search	Putting all integrations into shell
`MCP`	External capability protocol	Let Claude access GitHub, Sentry, databases	Connecting too many servers, tool definitions overwhelming context
`Plugin`	Packaging and distribution layer	Distribute skills/hooks/MCP together	Treating plugin as a runtime primitive
`Skill`	On-demand loaded knowledge/workflow	Give Claude a "method package"	Skill trying to be both encyclopedia and deployment script
`Hook`	Rule-enforcing interception layer	Execute rules around lifecycle events	Using hooks to replace all model judgment
`Subagent`	Context-isolated work unit	Parallel research, limit tools and permissions	Unbounded fan-out, governance loss

Plan Mode — подтверждение перед выполнением

Plan Mode разделяет исследование и выполнение. Исследование работает в режиме «только чтение», файлы меняются только после подтверждения плана. Для сложных рефакторингов, миграций и кросс-модульных изменений это существенно лучше, чем немедленное редактирование. Вход: двойной `Shift+Tab`.

---

4. Проектирование Skills: рабочие процессы, а не библиотека шаблонов

Skills лучше всего понимать как **пакеты рабочих процессов по запросу**. Их дескрипторы остаются в контексте, а полное тело загружается только при необходимости.

Признаки хорошего Skill

- Описание говорит модели, **когда** использовать, а не только что внутри.

- Определяет полные шаги, входы, выходы и условия остановки.
- Тело содержит навигацию и ключевые ограничения; большие справочники — в отдельных файлах.
- Skills с побочными эффектами должны явно запрещать автовызов моделью (`disable-model-invocation: true`).

Три типичных типа Skills

Тип 1: Чеклист (качественный гейт) — запускать перед релизом:

```
name: release-check
description: Use before cutting a release.

## Pre-flight (всё должно пройти)
- [ ] cargo build --release
- [ ] cargo clippy -- -D warnings
- [ ] Версия обновлена в Cargo.toml
- [ ] CHANGELOG обновлён
- [ ] kaku doctor проходит на чистом окружении
```

Тип 2: Workflow (стандартизированные операции) — с явным вызовом и откатом:

```
name: config-migration
disable-model-invocation: true

## Шаги
1. Backup: cp config.toml config.toml.bak
2. Dry run: kaku config migrate --dry-run
3. Apply: убрать --dry-run после проверки
4. Verify: kaku doctor

## Откат
cp config.toml.bak config.toml
```

Тип 3: Доменный эксперт — сбор доказательств по фиксированному пути вместо угадывания (матрица решений: симптом → первая проверка → формат вывода).

Оптимизация дескрипторов

Каждый включённый скилл держит дескриптор в контексте. Разница огромна:

```
# Неэффективно (~45 токенов)
description: |
This skill helps you review code changes in Rust...

# Эффективно (~9 токенов)
description: Use for PR reviews with focus on correctness.
```

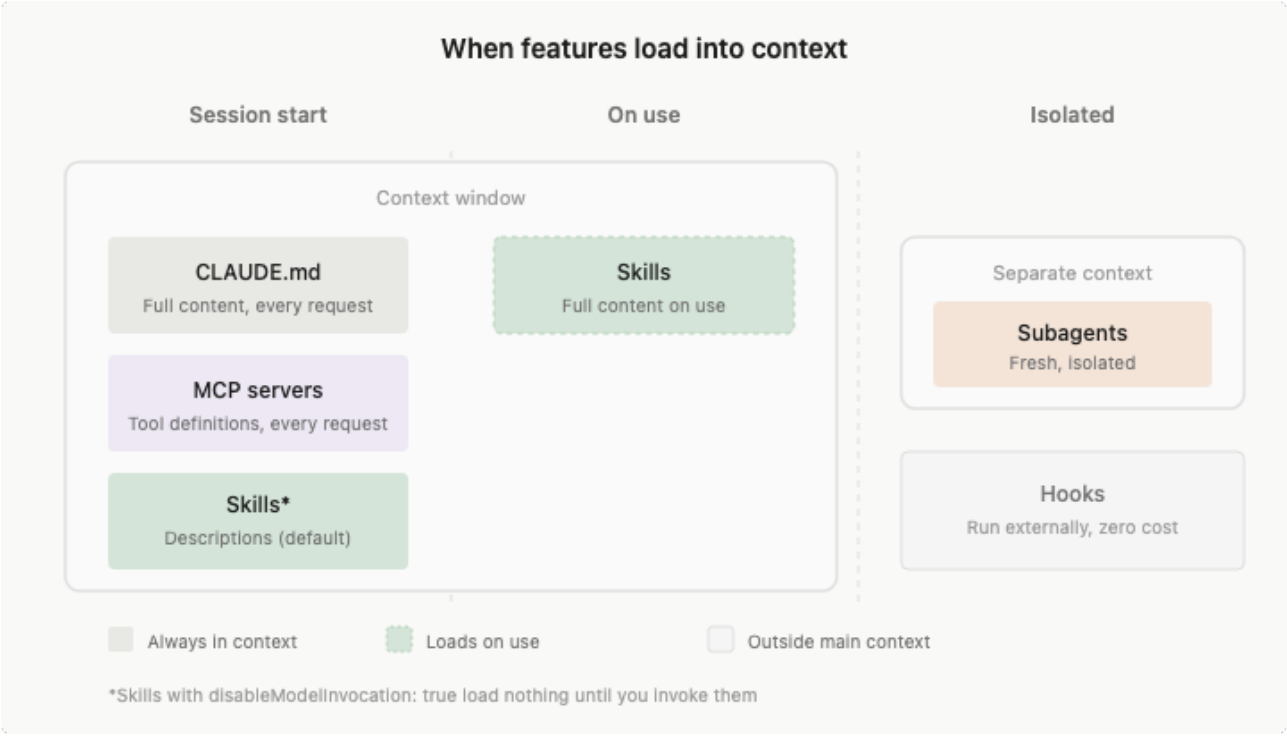
Стратегия вызова:

- Высокая частота (>1/сессию) → автовызов, оптимизировать дескриптор
- Низкая частота (<1/сессию) → отключить автовызов, вызывать вручную
- Очень низкая (<1/месяц) → удалить скилл, задокументировать в AGENTS.md

---

5. Проектирование Tools: помочь Claude выбрать правильный инструмент

Инструменты для Claude и API для людей должны проектироваться по-разному. API для людей оптимизируют полноту функций. Инструменты для агентов должны оптимизировать **правильный выбор и правильное использование**.

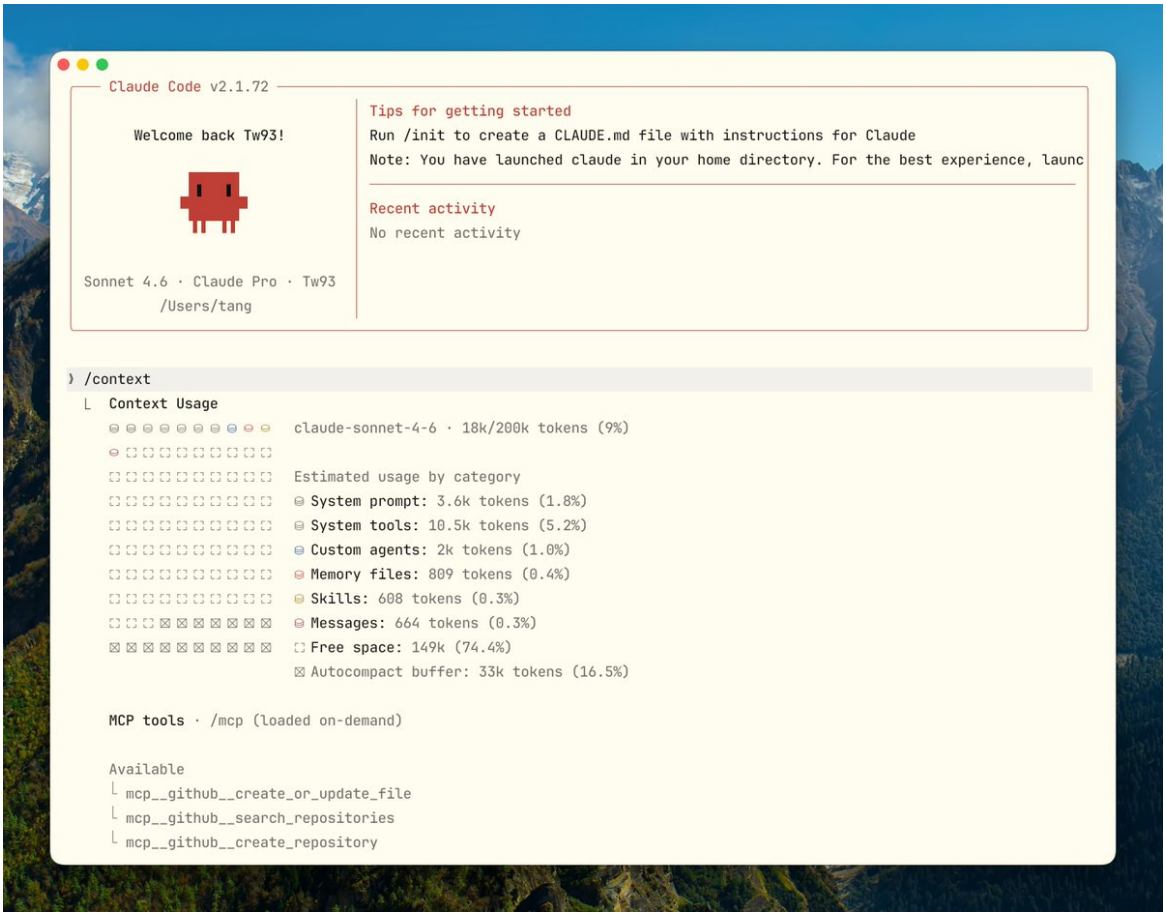


Нахождение оптимальной точки проектирования инструментов

Принципы проектирования:

- Префиксы по системе/ресурсу: `github_pr_*`, `jira_issue_*`
- Поддержка `response_format: concise / detailed` для больших ответов
- Ошибки должны содержать подсказки по исправлению, а не только коды
- Не экспортируйте много низкоуровневых фрагментов; объединяйте в задачи высокого уровня

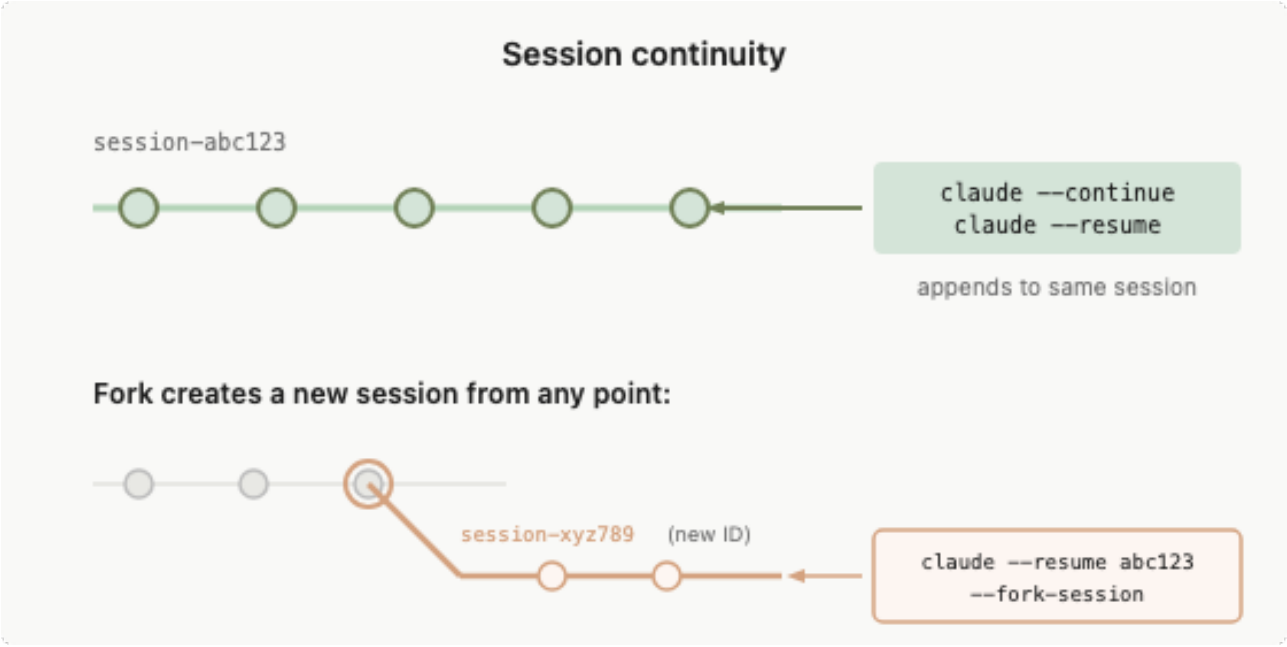
Эволюция AskUserQuestion



AskUserQuestion как выделенный инструмент — три подхода

Anthropic пробовали три подхода к тому, как агент должен останавливаться и задавать вопрос пользователю: (1) параметр question в существующих инструментах — Claude его игнорировал; (2) специальный markdown-формат — хрупкий, нет жёсткого контроля; (3) **выделенный инструмент AskUserQuestion** — значительно стабильнее.

Практический вывод: если хотите, чтобы Claude остановился и спросил — дайте ему выделенный инструмент. Флаги и конвенции формата модель легко пропускает.



Эволюция Todo Tool — контроли ревизуются по мере роста модели

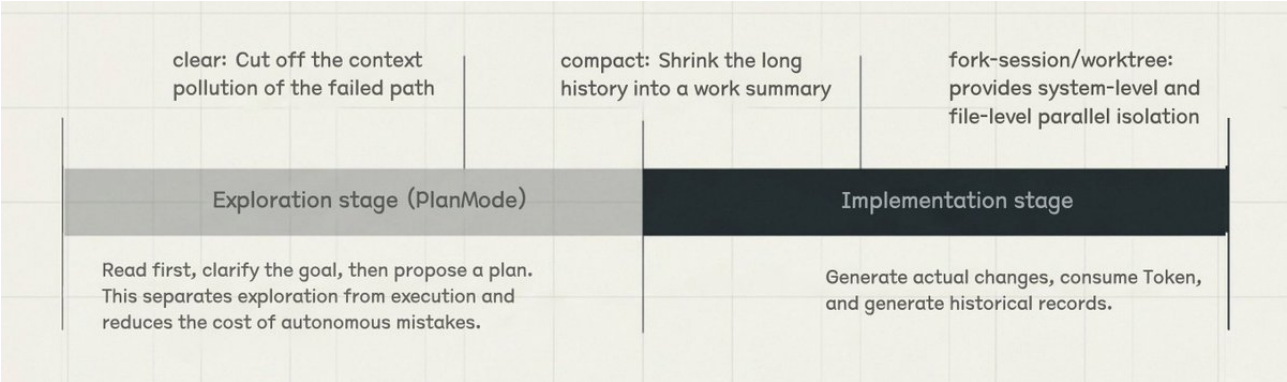
⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Принцип «выделенный инструмент для каждого типа действия» — фундаментален для агентных пайплайнов. Если в вашем пайплайне агент должен передать задачу на следующую стадию, сделайте это явным tool call, а не просто текстовым выводом. Это даёт программно-проверяемый сигнал перехода.

---

6. Hooks: ваш код до и после действий Claude

Hooks — это способ вынести работу из суждений модели «на лету» в **детерминированные процессы**: должно ли запуститься форматирование, можно ли менять защищённые файлы, нужно ли уведомить после завершения.

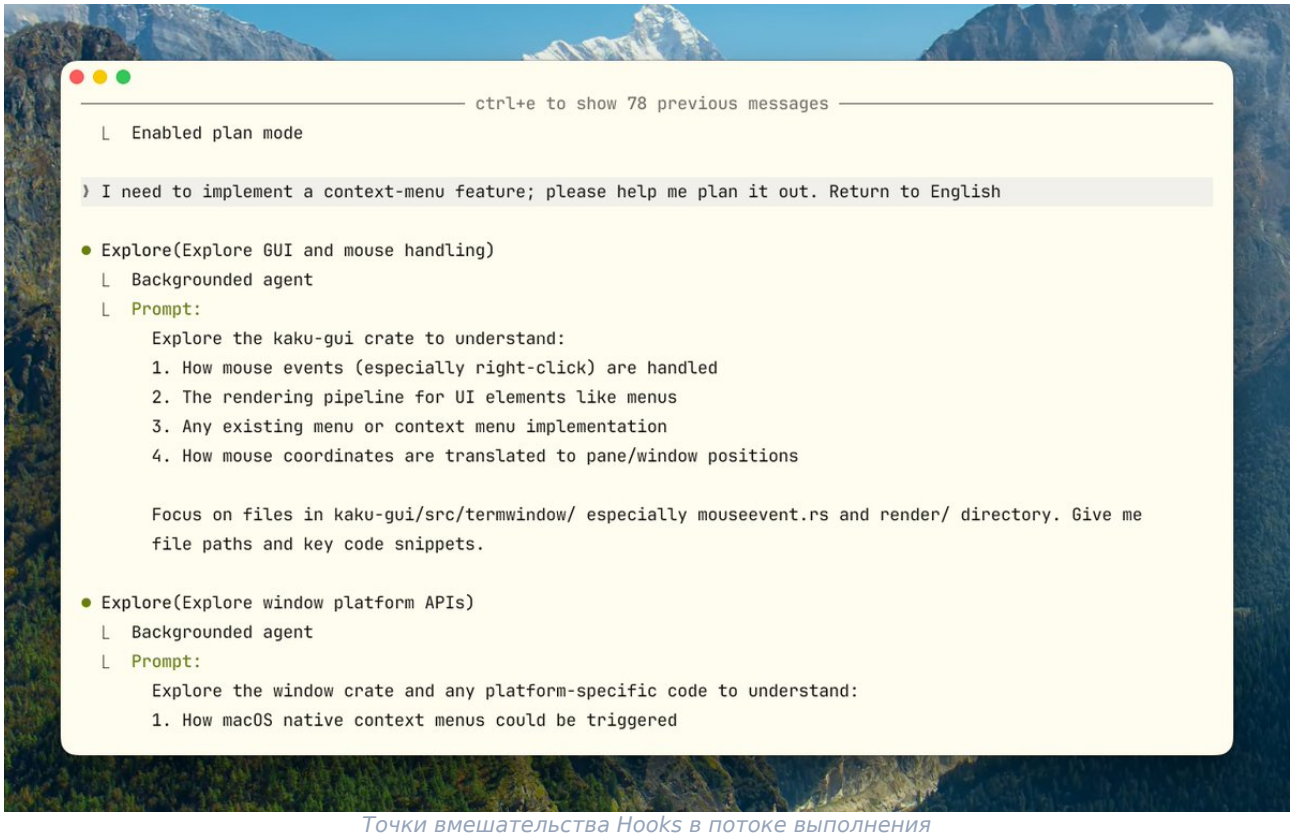


Конфигурация Hooks — точки перехвата

Пример: проверка Rust после каждого редактирования:

```
{
"hooks": {
"PostToolUse": [{
"matcher": "Edit",
"pattern": "*.rs",
"hooks": [{
```

```
"type": "command",
"command": "cargo check 2>&1 | head -30",
"statusMessage": "Running cargo check..."
}]
}]
}
}
```



Точки вмешательства Hooks в потоке выполнения

В сессии на 100 редактирований, экономия 30–60 секунд каждый раз даёт 1–2 часа. **Ограничивайте длину вывода** (`| head -30`), чтобы вывод хука не загрязнял контекст.

Трёхслойный стек: Hooks + Skills + CLAUDE.md

- **CLAUDE.md:** декларирует «тесты и lint должны пройти перед коммитом»
- **Skill:** объясняет, в каком порядке запускать тесты, как читать ошибки, как чинить
- **Hook:** жёсткая валидация на критических путях, блокировка при необходимости

Один слой всегда имеет пробелы. Все три вместе — это то, что реально работает.

---

7. Субагенты: изолированные исполнители

Субагент — независимый экземпляр Claude с собственным контекстным окном и только разрешёнными инструментами. Главная ценность — **изоляция**, а не параллелизм. Сканирования, тесты, ревью — всё что генерирует большой объём вывода — уходит в субагент, а основной поток получает только резюме.

Claude Code имеет встроенные типы: **Explore** (read-only, Haiku для экономии), **Plan** (планирование), **General** (общий) и custom.

Явные ограничения в конфигурации:

- `tools / disallowedTools`: ограничить набор инструментов; не давать те же широкие права, что и основному потоку
- `model`: для исследования — Haiku/Sonnet, для важных ревью — Opus
- `maxTurns`: предотвратить неконтролируемый запуск
- `isolation: worktree`: изолировать файловую систему при изменении файлов

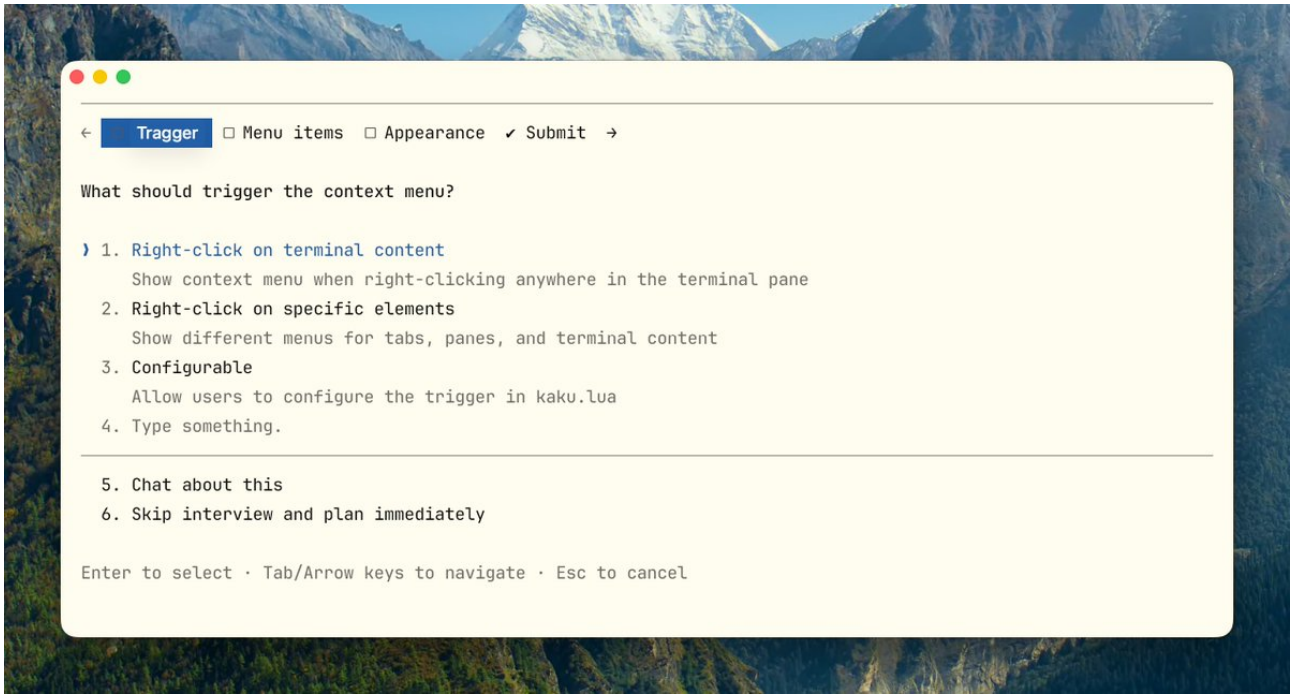
Типичные антипаттерны субагентов:

- Субагент с теми же широкими правами, что и основной поток — изоляция теряет смысл
- Формат вывода не зафиксирован — основной поток не может использовать результат
- Сильные зависимости между подзадачами с частым обменом промежуточным состоянием — не подходит для субагентов

---

8. Prompt Caching как архитектурное ограничение

> *Prompt caching — это не просто оптимизация стоимости. Он формирует архитектуру. Высокий cache hit rate снижает стоимость, улучшает задержку и делает практичными более щедрые rate limits.*



Оптимизация расположения промпта для кеширования

Кеширование промптов работает через **prefix matching** — содержимое от начала запроса до каждой точки cache_control кешируется. Поэтому порядок важен:

```
Порядок промпта Claude Code:
1. System Prompt → Статический, заблокирован
2. Tool Definitions → Статический, заблокирован
```

```
3. Chat History → Динамический, идёт после
4. Текущий ввод → Последний
```

Типичные ошибки кеширования:

- Временные метки в статическом system prompt (меняется каждый раз)
- Недетерминированная перетасовка порядка определений инструментов
- Добавление/удаление инструментов в середине сессии

Не переключайте модели посреди сессии

Кеш промпта привязан к модели. Если вы провели 100K токенов с Opus и хотите задать простой вопрос, **переключение на Haiku дороже**, чем продолжение с Opus, потому что кеш нужно заново строить для Haiku. Если нужно переключиться — передайте через субагент: Opus готовит «handoff message» для другой модели.

Dimension	Good Tools	Bad Tools
Name	`jira_issue_get`, `sentry_errors_search`	`query`, `fetch`, `do_action`
Parameters	`issue_key`, `project_id`, `response_format`	`id`, `name`, `target`
Return	Information directly relevant to next decision	UUIDs, internal fields, raw noise
Scope	Single purpose, clear boundaries	Multiple actions mixed, opaque side effects
Cost	Default output controlled, truncatable	Default returns oversized context
Errors	Include correction suggestions	Only return opaque error codes

Механизм компрессии (compaction) — форкинг и сжатие контекста

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Для multi-agent пайплайнов это имеет прямое следствие: не используйте одну сессию Claude Code для всех стадий (Design → Dev → QA). Каждая стадия — новая сессия или субагент, передача состояния через HANDOFF.md / структурированные артефакты. Это сохраняет cache hit rate высоким и контекст чистым.

9. Цикл верификации: нет верификатора — нет инженерного агента

«Claude говорит, что готово» имеет мало инженерной ценности. Важно знать: **правильно ли**, **можно ли откатить**, **аудитируемо ли**.

Уровни верификации:

- **Базовый:** exit codes, lint, typecheck, unit test
- **Средний:** интеграционные тесты, сравнение скриншотов, contract test, smoke test
- **Высокий:** проверка production логов, метрики мониторинга, ручные чеклисты

Явно определяйте верификацию в промпте, Skill и CLAUDE.md:

```
## Verification

Для backend-изменений:
- Запустить make test и make lint
- Для API-изменений: обновить contract tests в tests/contracts/

Для UI-изменений:
- Скриншоты до/после

Definition of done:
- Все тесты проходят
- Lint проходит
- Нет TODO без явного трекинга
```

Простой тест: если вы не можете чётко сформулировать «как Claude узнает, что сделал правильно» — задача вероятно не готова для автономного выполнения.

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

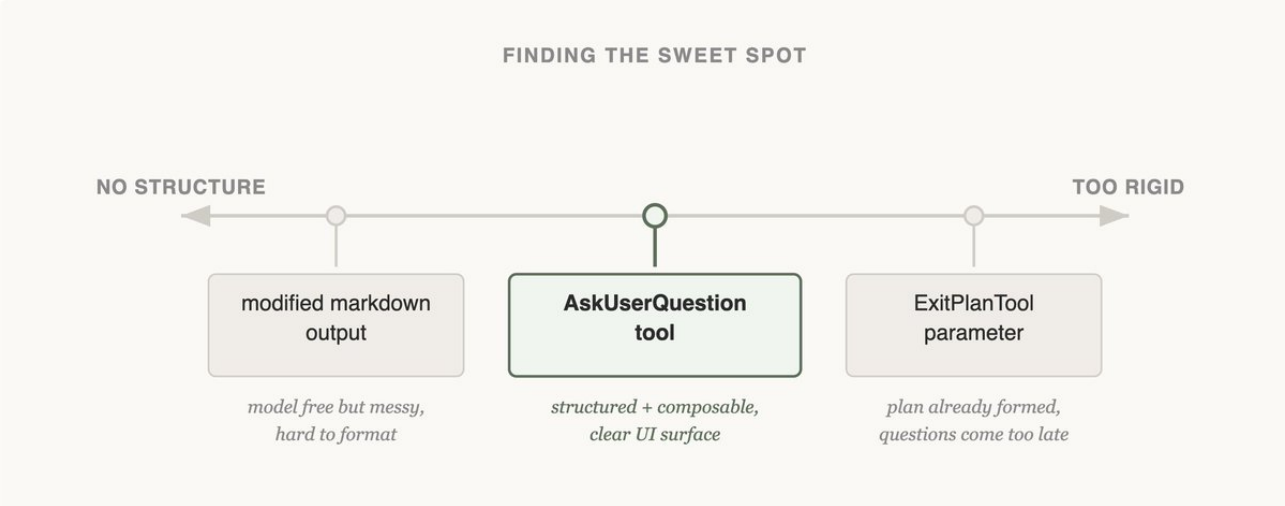
В формальном агентном пайплайне верификация — это гейт между стадиями. QA-агент не просто «проверяет» — он выдаёт машинно-читаемый verdict (PASS/FAIL + детали). Рекомендация: определите Verification Protocol для каждого типа задачи в вашем SDLC и привяжите к нему конкретные команды, пороги, и условия отката.

10. Полезные команды

Управление контекстом:

```
/context # Инспекция расхода токенов, включая MCP и файлы
/clear # Сбросить сессию; полезно когда та же ошибка исправляется второй раз
/compact # Сжать с сохранением ключевых точек; работает лучше с Compact Instructions
/memory # Проверить какой CLAUDE.md реально загружен
```

Возможности и управление:



Команда /mcp — статус подключения серверов и расход токенов

```
/mcp # Управление MCP-подключениями, проверка расхода токенов
/hooks # Управление хуками — ключевая точка контроля
/permissions # Просмотр/обновление whitelist разрешений
/model # Переключение моделей: Opus/Sonnet/Haiku
```

Непрерывность сессий:

```
claude --continue # Возобновить последнюю сессию в текущей директории
claude --resume # Выбрать историческую сессию для возобновления
claude --continue --fork # Форкнуть сессию для другого подхода
claude --worktree # Создать изолированный git worktree
claude -p "prompt" # Неинтерактивный режим для CI / скриптов
claude -p --output-format json # Структурированный вывод для скриптов
```

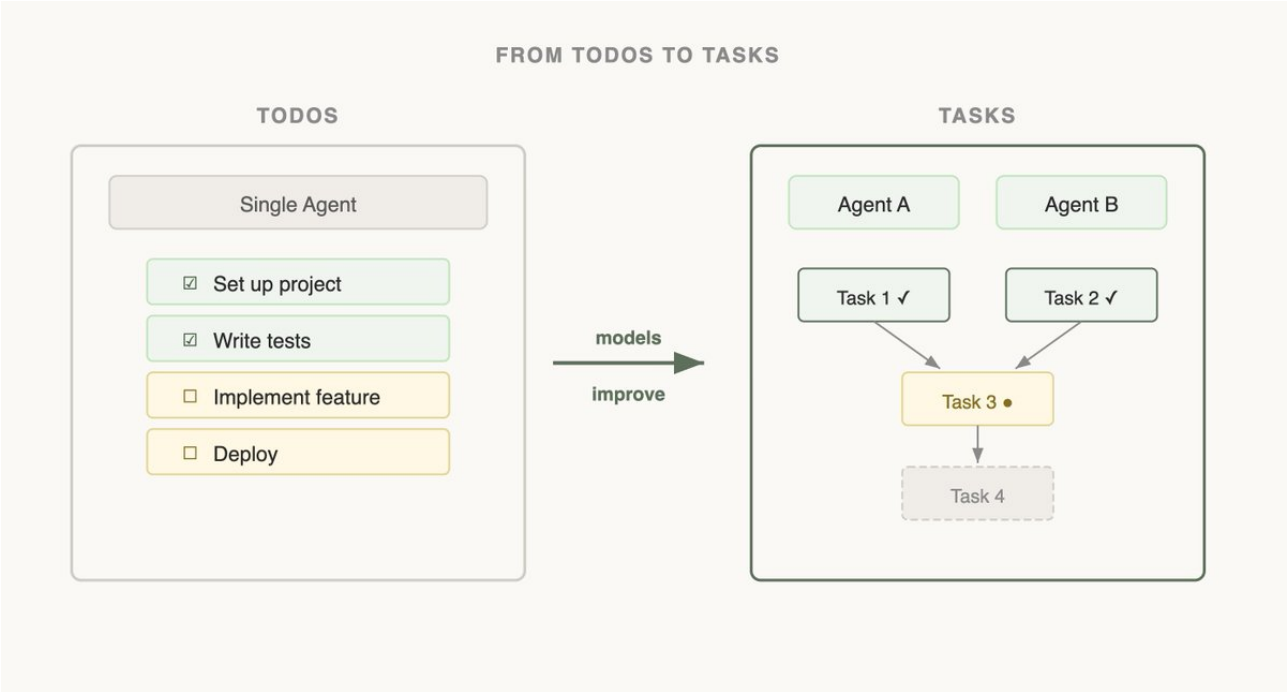
Менее известные, но полезные:

- `/simplify` — быстрый проход по недавно изменённому коду (качество, переиспользование)
- `/rewind` — возврат к раннему чекпоинту сессии + новая суммаризация
- `/btw` — быстрый побочный вопрос без прерывания основной задачи
- `/insight` — анализ текущей сессии для CLAUDE.md (паттерны, которые повторялись, но не записаны)
- **Двойной ESC** — возврат предыдущего ввода для редактирования (вместо перенабора)
- **Ctrl+B** — перевести длинную bash-команду в фон; Claude проверит результат позже
- История сессий хранится локально в `~/.claude/projects/`

---

11. Как писать хороший CLAUDE.md

Я думаю о CLAUDE.md как о **контракте сотрудничества** между вами и Claude. Это не командная документация и не база знаний. Кладите только то, что должно выполняться в каждой сессии.



«Будь проще» — принцип CLAUDE.md

Мой совет прост: начните с нуля. Используйте Claude Code сначала, затем добавляйте записи только когда заметите, что повторяете одну и ту же инструкцию. Для добавления — введите # чтобы дописать текущий разговор в CLAUDE.md.

Что включать:

- Команды build, test, lint, run (**самое важное**)
- Ключевая структура директорий и границы модулей
- Явные ограничения стиля кода и именования
- Неочевидные зависимости окружения и подводные камни
- Запреты и высокорисковые операции (NEVER list)
- Информация, которая должна пережить компрессию (Compact Instructions)

Что НЕ включать:

- Длинные вводные описания проекта
- Полную документацию API
- Расплывчатые принципы вроде «пиши качественный код»
- Очевидную информацию, которую Claude может вывести из чтения репозитория
- Большие справочные материалы и знания для редких задач (их — в skills)

Шаблон высокого качества:

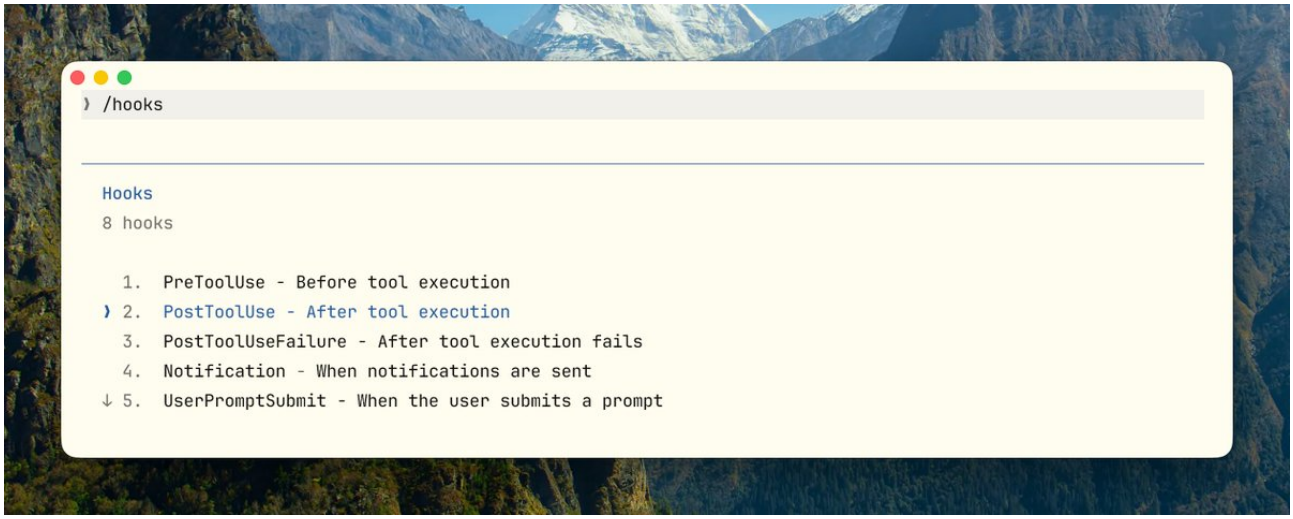
```
# Project Contract

## Build And Test
- Install: pnpm install
- Dev: pnpm dev
- Test: pnpm test
- Lint: pnpm lint

## Architecture Boundaries
- HTTP handlers: src/http/handlers/
- Domain logic: src/domain/
- Shared types: src/contracts/
- НЕ класть persistence logic в handlers

## NEVER
- Изменять .env, lockfiles, CI secrets без одобрения
- Удалять feature flags без поиска по всем call sites
- Коммитить без запуска тестов

## ALWAYS
- Показать diff перед коммитом
- Обновить CHANGELOG для user-facing изменений
```

Полезная привычка — обновлять CLAUDE.md сразу после исправления ошибки Claude: *«Обнови свой CLAUDE.md, чтобы больше не допускать эту ошибку.»* Claude обычно хорошо справляется с написанием таких правил для себя.

---

12. Практика на реальном проекте (Kaku)

Во время каникул на китайский Новый год я построил open-source терминал Kaku с Claude Code. Он использует Rust и Lua в ядре, плюс кастомную систему конфигурации.



Структура проекта Kaku — организация Claude Code конфигурации

«Прозрачность окружения» важнее, чем кажется

Claude Code вызывает реальные шеллы, git, пакетные менеджеры. Как только один слой непрозрачен — модель начинает угадывать, и надёжность падает. Добавление команды `doctor`, собирающей состояние окружения, зависимостей и конфигурации, устраняет множество случаев неправильного понимания окружения.

Hooks для смешанных языков

```
"PostToolUse": [
{"matcher": "Edit", "pattern": "*.rs",
"hooks": [{"type": "command",
"command": "cargo check 2>&1 | head -30"}]},
{"matcher": "Edit", "pattern": "*.lua",
"hooks": [{"type": "command",
"command": "luajit -b $FILE /dev/null 2>&1 | head -10"}]}
]
```

Эталонная структура проекта:

```
Project/
├── CLAUDE.md
├── .claude/
│ ├── rules/ # core.md, config.md, release.md
│ ├── skills/ # runtime-diagnosis/, config-migration/,
│ │ # release-check/, incident-triage/
│ ├── agents/ # reviewer.md, explorer.md
│ └── settings.json
└── docs/ai/ # architecture.md, release-runbook.md
```

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Для вашего проекта с Django/React стеком рекомендуется аналогичная структура: .claude/rules/ — отдельные правила для Python (Django) и TypeScript (React), .claude/skills/ — миграции, деплой, тестирование API. В CLAUDE.md — только команды сборки, границы модулей и NEVER/ALWAYS списки. Если вы используете несколько проектов — держите личные defaults в ~/.claude/, а проектные различия — в каждом .claude/ проекта.

13. Типичные антипаттерны

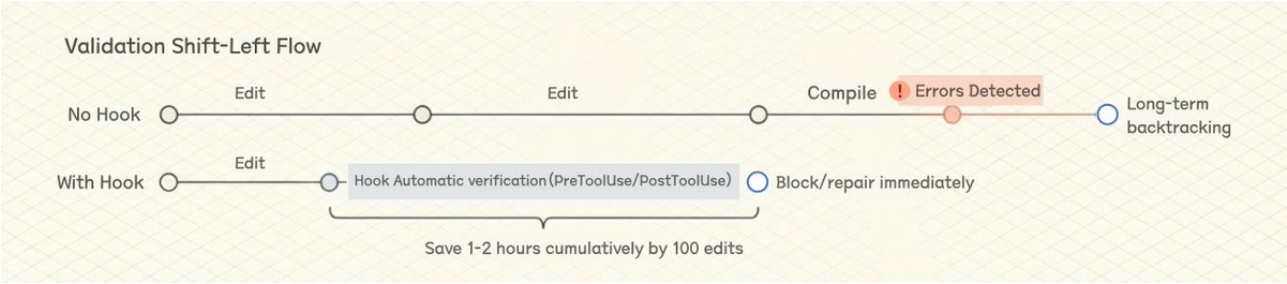
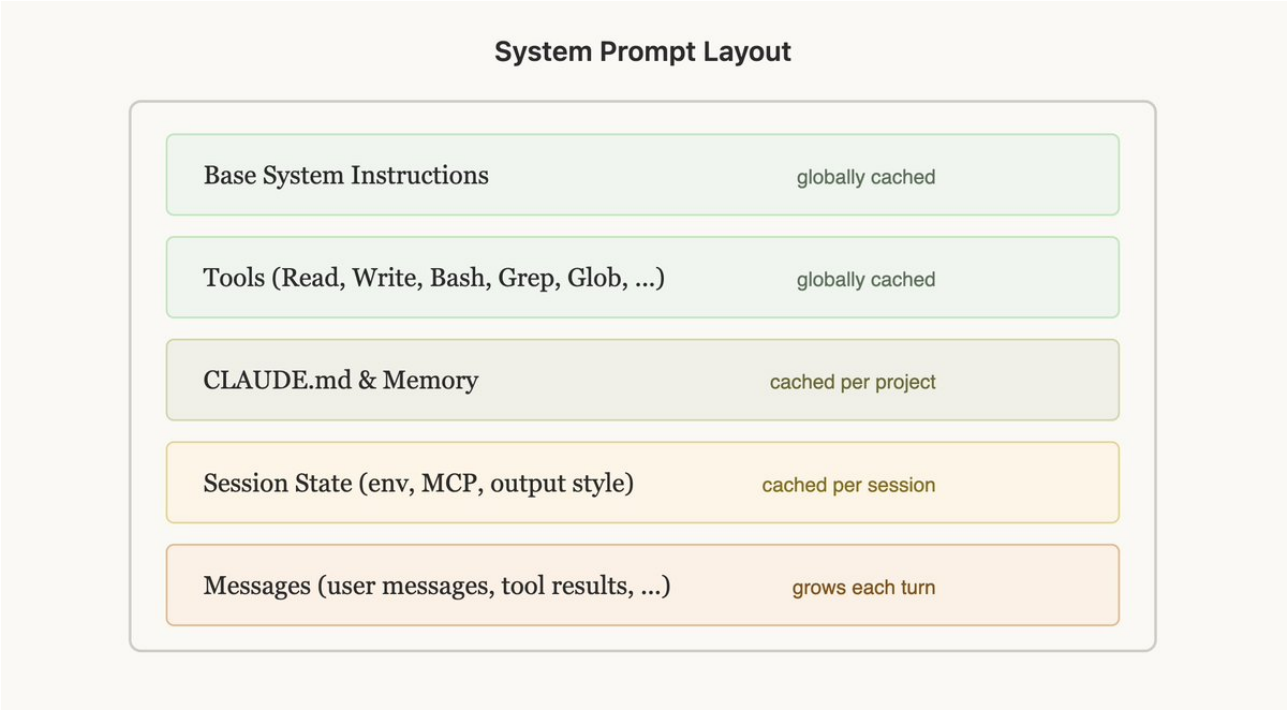


Таблица антипаттернов — симптомы и исправления

Антипаттерн	Симптом	Исправление
CLAUDE.md как вики	Контекст загрязнён, инструкции размыты	Только контракт; материалы → skills/rules
Skill-свалка	Описание не триггерит стабильно	Один скилл = одна задача
Слишком много tools	Неправильный выбор, schema переполняет контекст	Объединить, чёткий namespacing
Нет верификации	Claude может только «чувствовать что готово»	Привязать verifier к каждому типу задачи
Чрезмерная автономия	Бесконтрольный multi-agent параллелизм	Минимизировать роли и права; maxTurns
Нет сегментации контекста	Ресёрч, имплементация, ревью в одном потоке	/clear для смены задачи; субагент для тяжёлого
Накопление разрешений	rm -rf в settings.json	Регулярно ревизовать allowedTools

---

14. Проверка здоровья конфигурации



Health Check — оценка конфигурации Claude Code

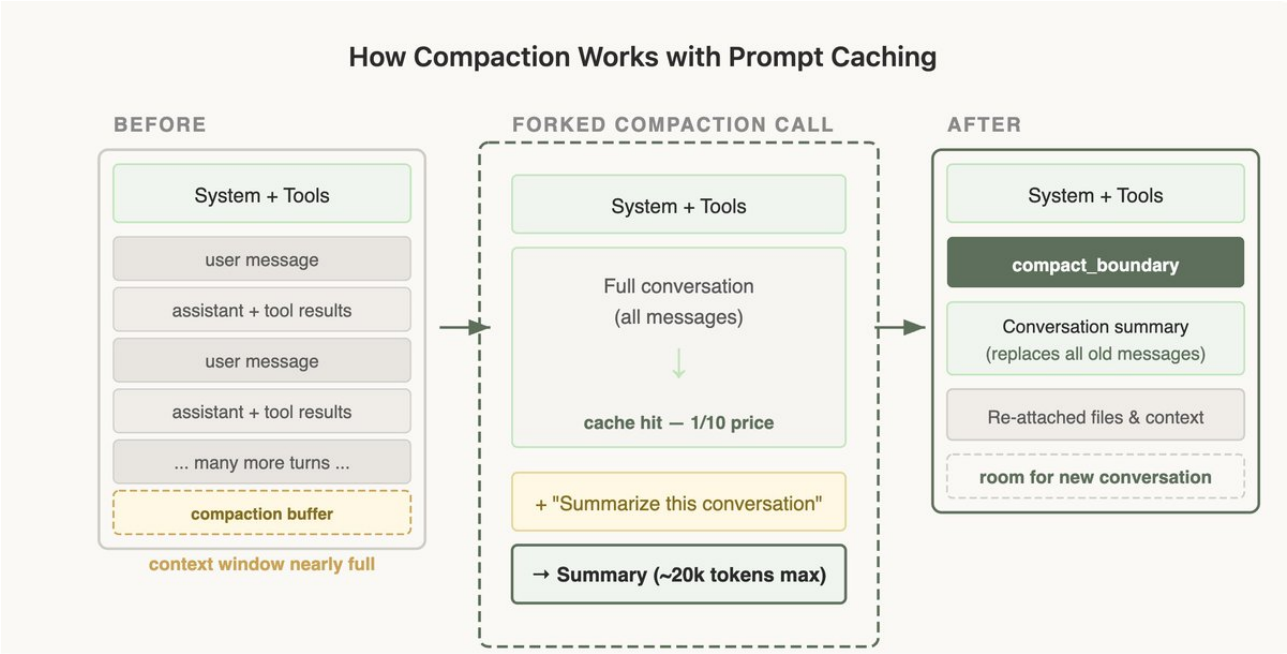
На основе шестислойной модели из этого handbook, автор упаковал проверки в open-source проект:

```
npx skills add tw93/claude-health
```

После установки запустите `/health` в любой сессии. Он оценит сложность проекта, проверит CLAUDE.md, rules, skills, hooks, allowedTools и поведенческие паттерны, затем выдаст приоритизированный отчёт: исправить сейчас / структурные проблемы / постепенные улучшения.

---

15. Заключение



Три стадии использования Claude Code

Стадия	Фокус	Восприятие эффективности
Пользователь инструмента	«Как использовать эту фичу?»	Полезно, но ограниченно
Оптимизатор процессов	«Как сделать сотрудничество плавнее?» CLAUDE.md + Skills	Значительное улучшение
Проектировщик систем	«Как сделать, чтобы агент работал автономно в рамках ограничений?»	Качественное изменение

На третьей стадии вопрос меняется с «как использовать эту фичу?» на «как сделать, чтобы агент работал автономно в рамках ограничений?» Это фундаментально разные задачи.

Один практический тест важнее большинства: если вы не можете чётко сформулировать, как выглядит «готово» — задача вероятно не готова для автономного выполнения Claude. Без критериев приёмки нет надёжного понятия правильного ответа, какой бы мощной ни была модель.

⚠ Дополнение Claude Opus 4.6 (Март 2026)

Путь от «пользователя инструмента» к «проектировщику систем» — это именно переход от ad-hoc использования к формальному процессу (SDLC). Для нетехнических основателей, управляющих AI-агентами: ваша главная задача — не писать промпты, а проектировать систему ограничений, верификации и передачи состояния. CLAUDE.md — это ваш Product Requirements Document для агента. Skills — это ваши SOP. Hooks — это ваши автоматические quality gates. Субагенты — это ваша команда с чёткими ролями.

Оригинал: https://tw93.fun/en/2026-03-12/claude.html
Автор: Tw93 — https://x.com/HiTw93
Подготовил перевод: alienxs2 — https://github.com/alienxs2/cc_bestpractice_russian

Перевод и дополнения: Claude Opus 4.6 (Anthropic), Март 2026